Relatório de Integração e Diagnóstico de Falhas: T800

De: Guilherme Andrade | **Para:** Suporte Técnico EngineAI

Este documento descreve o estado atual do projeto de integração do robô T800, apresentando o passo a passo da configuração do ambiente de simulação e reportando os problemas comportamentais encontrados na execução do modelo no MuJoCo.

1. Especificações do Sistema e Ambiente

Sistema Operacional	Linux Mint (File System: ext4)
Rede	192.168.0.0/24 (Robô configurado em 192.168.0.162)
Ambiente Docker	Modo de rede `host` (Setup dinâmico via t800_docker_setup.tar.gz)
Stack de IA (Python)	Gymnasium (mujoco envs) e Stable-Baselines3 (PPO - MlpPolicy)
Workspaces EngineAI	Native SDK (t800.xml) e ROS2 Workspace

2. Guia de Configuração e Execução (Estado Atual)

O fluxo abaixo foi consolidado para inicializar o ambiente, compilar o MuJoCo, configurar as dependências de Python (RL) e estabelecer a telemetria com o PlotJuggler:

Passo 1: Inicialização do Ambiente Docker

O processo é feito isoladamente dentro do contêiner oficial para evitar conflitos com o sistema operacional hospedeiro. Navega-se até a pasta raiz do SDK para gerar e acessar o ambiente.

Passo 2: Compilação do Native SDK e ROS2

O script de build raiz é executado no contêiner para compilar o sistema central e gerar as interfaces de mensagens (msg) do ROS2, essenciais para a telemetria no PlotJuggler.

Passo 3: Compilação e Instalação do Mujoco

O motor do simulador é construído através dos scripts nativos da EngineAI. Adicionalmente, as bibliotecas Python (dependências para o projeto RL) são instaladas para a ponte com o simulador.

Passo 4: Configuração de Execução (Modo Simulação)

Edição do arquivo `assets/config/t800/mode.yaml` para garantir a configuração correta do ambiente virtual, alertando o sistema de controle de que não se trata do hardware físico.

Passo 5: Execução Integrada (Terminal 1 - Mujoco)

Com as variáveis de ambiente de rede do ROS2 rigorosamente configuradas, o simulador é executado para inicializar a simulação e exportar os dados vitais do robô.

Passo 6: Visualização de Dados (Terminal 2 - PlotJuggler)

Em um novo terminal com o mesmo ambiente, o PlotJuggler é iniciado. Seleciona-se o *"ROS2 Topic Subscriber"* carregando o layout de visualização `scripts/plotjuggler/common_data_display.xml` para ler torque e IMU em tempo real.

Passo 7: Controle da Máquina de Estados (Terminal 3)

O controle virtual (Gamepad) é iniciado para tirar o robô do estado de segurança *idle*. A interface é ativada e comandos de teclado (ex: Q + J) alternam o estado do robô para `pd_stand`.

3. Relatório de Falhas de Simulação (Issue Report)

Apesar de o ambiente estar perfeitamente configurado conforme o guia acima, foram identificados problemas comportamentais críticos no robô T800 dentro do MuJoCo durante o treinamento das políticas de Reinforcement Learning:

- **Falha na Estabilização (Stand-up Failure):** O modelo T800 exibe severas dificuldades estruturais para se levantar e manter uma postura ereta e estável. Na maioria das tentativas, a simulação sofre término precoce devido ao centro de massa atingir um limite muito baixo.
- **Comandos Faltantes (Missing Commands):** A interface de controle disponibilizada para a simulação carece de comandos de junta específicos ou parâmetros de equilíbrio que deveriam estar mapeados, essenciais para que o controlador consiga estabilizar as pernas e o tronco de forma efetiva.

Solicitação à EngineAI: Gostaria de verificar com a equipe de engenharia se existe algum modelo atualizado do MuJoCo, algum *patch* para adicionar as atuações de controle ausentes, ou se existem recomendações específicas de ganhos PD e limites de atuadores no arquivo `t800.xml` que precisem ser ajustados para solucionar o problema de estabilização.